Practica Optimización para Grandes volumenes de datos

Rodrigo Aguirregabiria, Yago Boleas, Adrian Estoquera

March 20, 2026

Table of Contents

1 Primeros pasos con Apache Kafka **3**
1.1 Acceso al entorno y Gestión de Tópicos 3
1.2 Flujo de mensajes: Producer y Consumer 4

2 Data streaming usando kafka-python **6**
2.1 Ejecución del código de los *Jupyter notebooks* 6
2.2 Gráfico generado por el consumidor 7
2.3 Efecto de la propiedad *auto_offset_reset* 7
2.4 Intentando enviar datos no serializables 8
2.5 Modificar el tópico para que tenga 2 particiones 9
2.6 Crear varios productores 10
2.7 Consumidores con ID común 13

3 Integrando procesos ML en data streaming **15**
3.1 Carga del modelo e inferencia 15
3.2 Envio de predicciones a un nuevo tópico 16

1 Primeros pasos con Apache Kafka

En esta sección se explica cómo se ha resuelto el primer ejercicio. El primer paso consiste en acceder al contenedor donde reside el servicio de Kafka para ejecutar las herramientas de línea de comandos integradas.

1.1 Acceso al entorno y Gestión de Tópicos

Para comenzar la interacción, nos conectamos al contenedor mediante el comando `docker exec`, lo que nos permite abrir una terminal interactiva dentro del nodo de Kafka:

Terminal — bash

```bash
docker exec -it kafka-jupyter bash
```

Una vez dentro, el objetivo es crear nuestro primer canal de comunicación o "Topic". Los tópicos son las categorías o nombres de feeds donde se almacenan y publican los registros de datos.

1.1.1 Creación y Verificación

Utilizamos el script `kafka-topics.sh` para definir una nueva estructura llamada `firstTopic`. Los comandos ejecutados y su función se resumen en la siguiente tabla:

Parámetro/Flag	Descripción
`-bootstrap-server`	Dirección del servidor Kafka inicial para establecer conexión.
`-create`	Instrucción para generar un nuevo tópico.
`-topic`	Especifica el nombre identificador (firstTopic).
`-list`	Lista todos los tópicos existentes en el servidor.
`-describe`	Muestra metadatos detallados (particiones, réplicas, etc.).

Table 1: Parámetros utilizados en la gestión de tópicos.

A continuación, se muestra la ejecución en la terminal de estas operaciones:

Terminal — bash

```
root@fa1dbb2ab5cd:/opt# ./kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --create --topic firstTopic
Created topic firstTopic.

root@fa1dbb2ab5cd:/opt# ./kafka/bin/kafka-topics.sh --list --bootstrap-server localhost:9092
firstTopic
prediction_topic
water_quality
```

Al ejecutar el comando de inspección (`–describe`), Kafka nos devuelve información sobre la topología del tópico:

Terminal — bash

```
root@fa1dbb2ab5cd:/opt# ./kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic firstTopic
Topic: firstTopic TopicId: rpAfjVfjRL2vNpCoVUfV_w PartitionCount: 1 ReplicationFactor: 1 Configs:
Topic: firstTopic Partition: 0 Leader: 0 Replicas: 0 Isr: 0
```

Análisis del resultado:

- **PartitionCount: 1**: El tópico no está dividido; todos los mensajes van a una única cola secuencial.
- **Leader: 0**: El nodo con ID 0 es el responsable de gestionar las lecturas y escrituras de esta partición.
- **Isr (In-Sync Replicas)**: Indica qué nodos están actualmente sincronizados con el líder (en este caso, solo el nodo 0).

1.2 Flujo de mensajes: Producer y Consumer

Para comprobar la arquitectura Pub/Sub (Publicador/Suscriptor) de Kafka, es necesario abrir dos terminales independientes dentro del contenedor.

1.2.1 Configuración del Suscriptor (Consumer)

Primero, ponemos a Kafka en modo escucha. El consumidor se suscribe al tópico `firstTopic` y queda a la espera de nuevos eventos:

Terminal — bash

```
root@fa1dbb2ab5cd:/opt# ./kafka/bin/kafka-console-consumer.sh
    --bootstrap-server localhost:9092 --topic firstTopic
```

1.2.2 Configuración del Publicador (Producer)

En una segunda terminal, iniciamos el publicador. Este script abre una línea de comandos interactiva donde cada línea de texto que escribamos se enviará como un mensaje individual al tópico:

Terminal — bash

```
root@fa1dbb2ab5cd:/opt# ./kafka/bin/kafka-console-producer.sh
    --broker-list localhost:9092 --topic firstTopic

Hola
Mundo
```

1.2.3 Verificación del resultado

Como se puede observar en las capturas adjuntas, existe una sincronización inmediata. Al escribir "Hola" y "Mundo" en la terminal del Producer, los mensajes aparecen instantáneamente en la terminal del Consumer. Esto demuestra que Kafka ha actuado como un intermediario eficiente, distribuyendo la información en tiempo real desde la fuente hacia los suscriptores interesados.

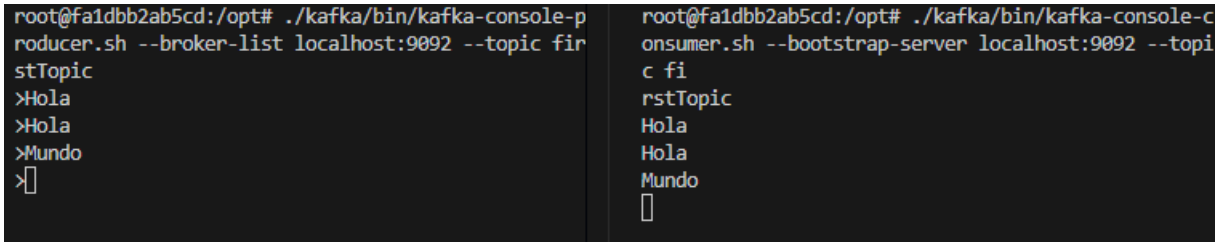


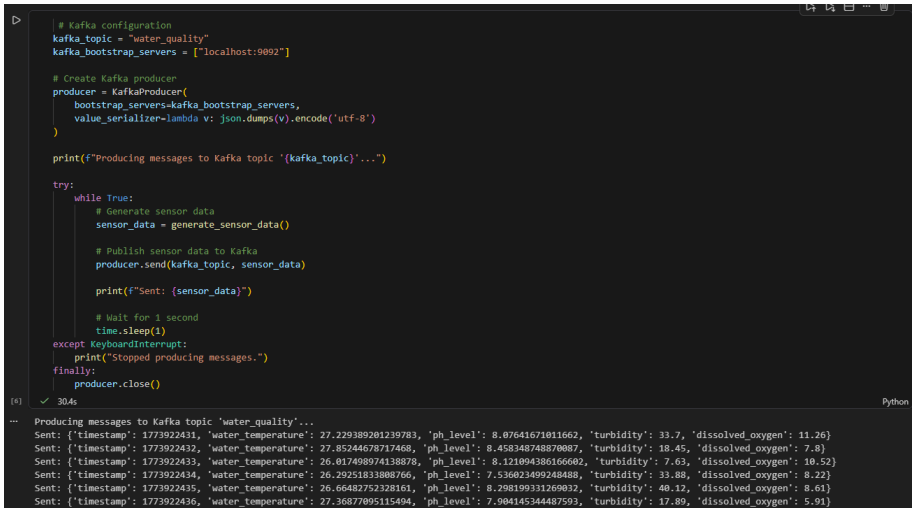
Figure 1: Muestra de dos terminales diferentes enviandose mensajes a través de un topic. A la izquierda el producto y a la derecha el consumidor.

2 Data streaming usando kafka-python

En la siguiente sección se muestra cómo se ha resuelto el 2º ejercicio.

2.1 Ejecución del código de los *Jupyter notebooks*

Si ejecutamos las celdas de los archivos proporcionados obtendremos lo siguiente:

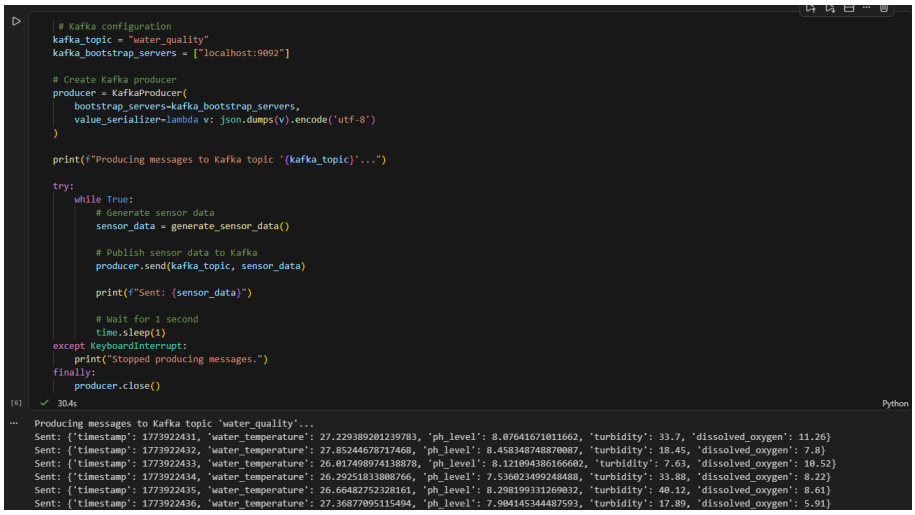


```
# Kafka configuration
kafka_topic = "water_quality"
kafka_bootstrap_servers = ["localhost:9092"]

# Create Kafka producer
producer = KafkaProducer(
    bootstrap_servers=kafka_bootstrap_servers,
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

print(f"Producing messages to Kafka topic '{kafka_topic}'...")

try:
    while True:
        # Generate sensor data
        sensor_data = generate_sensor_data()

        # Publish sensor data to Kafka
        producer.send(kafka_topic, sensor_data)

        print(f"Sent: {sensor_data}")

        # Wait for 1 second
        time.sleep(1)
except KeyboardInterrupt:
    print("Stopped producing messages.")
finally:
    producer.close()
```

[6] ✓ 30.4s Python

```
Producing messages to Kafka topic 'water_quality'...
Sent: {'timestamp': 1773922431, 'water_temperature': 27.229389201239783, 'ph_level': 8.07641671011662, 'turbidity': 33.7, 'dissolved_oxygen': 11.26}
Sent: {'timestamp': 1773922432, 'water_temperature': 27.85244678717468, 'ph_level': 8.458348748870087, 'turbidity': 18.45, 'dissolved_oxygen': 7.8}
Sent: {'timestamp': 1773922433, 'water_temperature': 26.017498974138878, 'ph_level': 8.121094386166602, 'turbidity': 7.63, 'dissolved_oxygen': 10.52}
Sent: {'timestamp': 1773922434, 'water_temperature': 26.29251833808766, 'ph_level': 7.536023499248488, 'turbidity': 33.88, 'dissolved_oxygen': 8.22}
Sent: {'timestamp': 1773922435, 'water_temperature': 26.66482752328161, 'ph_level': 8.298199331269032, 'turbidity': 40.12, 'dissolved_oxygen': 8.61}
Sent: {'timestamp': 1773922436, 'water_temperature': 27.36877095115494, 'ph_level': 7.904145344487593, 'turbidity': 17.89, 'dissolved_oxygen': 5.91}
```

Figure 2: Muestra del productor proporcionado enviando mensajes al topic *water_quality*.



```
# Kafka configuration
kafka_topic = "water_quality"
kafka_bootstrap_servers = ["localhost:9092"]

# Create Kafka producer
producer = KafkaProducer(
    bootstrap_servers=kafka_bootstrap_servers,
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

print(f"Producing messages to Kafka topic '{kafka_topic}'...")

try:
    while True:
        # Generate sensor data
        sensor_data = generate_sensor_data()

        # Publish sensor data to Kafka
        producer.send(kafka_topic, sensor_data)

        print(f"Sent: {sensor_data}")

        # Wait for 1 second
        time.sleep(1)
except KeyboardInterrupt:
    print("Stopped producing messages.")
finally:
    producer.close()
```

[6] ✓ 30.4s Python

```
Producing messages to Kafka topic 'water_quality'...
Sent: {'timestamp': 1773922431, 'water_temperature': 27.229389201239783, 'ph_level': 8.07641671011662, 'turbidity': 33.7, 'dissolved_oxygen': 11.26}
Sent: {'timestamp': 1773922432, 'water_temperature': 27.85244678717468, 'ph_level': 8.458348748870087, 'turbidity': 18.45, 'dissolved_oxygen': 7.8}
Sent: {'timestamp': 1773922433, 'water_temperature': 26.017498974138878, 'ph_level': 8.121094386166602, 'turbidity': 7.63, 'dissolved_oxygen': 10.52}
Sent: {'timestamp': 1773922434, 'water_temperature': 26.29251833808766, 'ph_level': 7.536023499248488, 'turbidity': 33.88, 'dissolved_oxygen': 8.22}
Sent: {'timestamp': 1773922435, 'water_temperature': 26.66482752328161, 'ph_level': 8.298199331269032, 'turbidity': 40.12, 'dissolved_oxygen': 8.61}
Sent: {'timestamp': 1773922436, 'water_temperature': 27.36877095115494, 'ph_level': 7.904145344487593, 'turbidity': 17.89, 'dissolved_oxygen': 5.91}
```

Figure 3: Muestra de como el consumidor proporcionado que escucha al topic *water_quality* recibe los mensajes del productor.

En las capturas puede obsrvarse cómo los mensajes que le llegan al consumidor son los mismos que se acaban de enviar por el productor.

2.2 Gráfico generado por el consumidor

El consumidor puede procesar los datos y realizar con ellos cualquier lógica de negocio. En este caso, los datos se envían a una función que realiza un *plot* que simula el comportamiento de un *dashboard*:

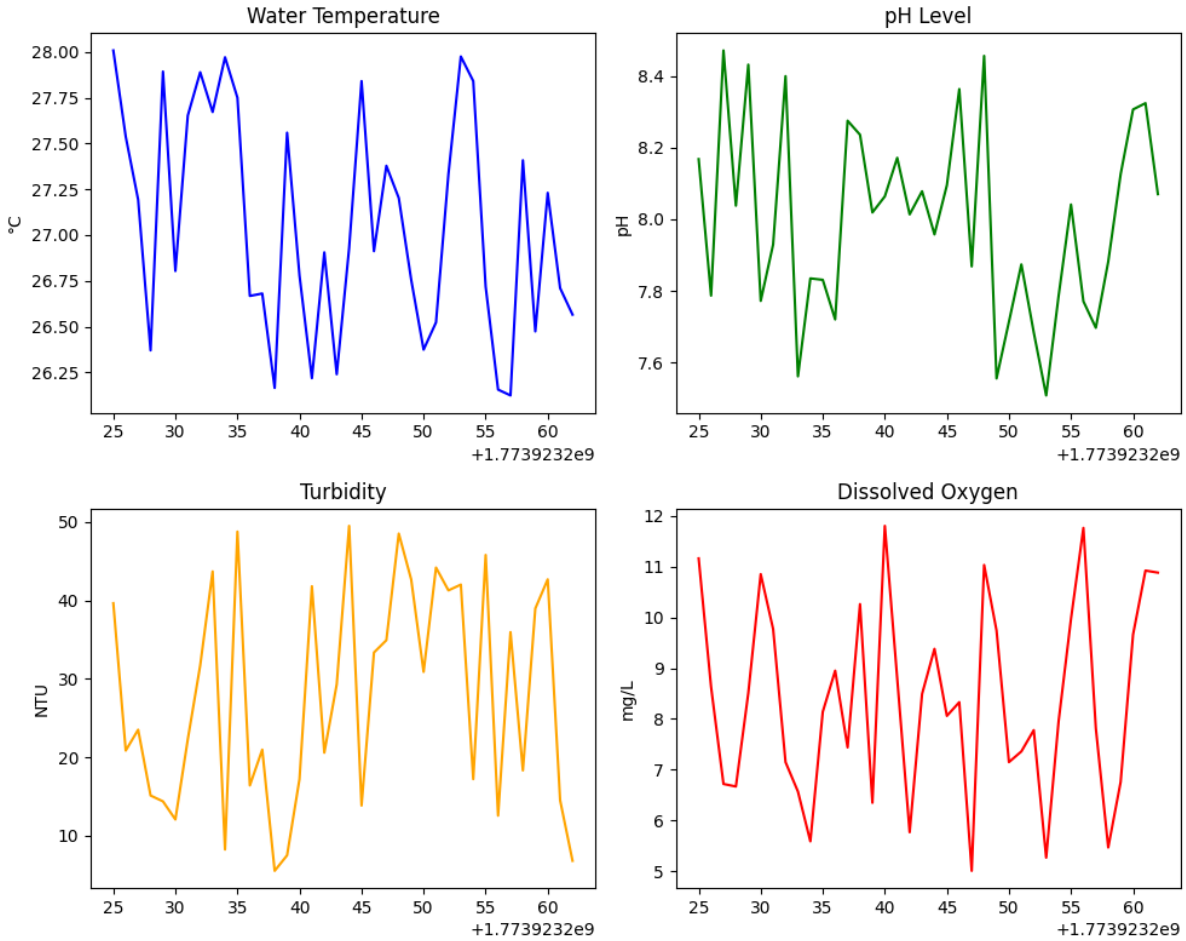


Figure 4: Imagen generada por el consumidor con los datos que recibe desde el topic *water_quality*.

2.3 Efecto de la propiedad *auto_offset_reset*

Esta propiedad determina qué debe hacer el consumidor cuando se conecta a un tópico de Kafka y es la primera vez que se conecta o el offset actual inválido (los datos se han borrado por la política de retención).

Si aplicamos la configuración *earliest* es consumidor retrocede al inicio del log y lee todos los mensajes históricos que estén disponibles por el broker. Es la configuración más útil cuando se desea evitar perder cualquier dato tras una caída del sistema.

La configuración *latest* ignora los mensajes antiguos y se queda escuchando solamente los que se produzcan desde su conexión. Es una configuración pensada para la monitorización puramente en tiempo real donde los datos del pasado ya no son relevantes para el sistema de procesamiento.

2.4 Intentando enviar datos no serializables

Podemos intentar enviar estructuras de datos complejas como *arrays* de *NumPy* de la siguiente manera:

Python

```python
# Create Kafka producer
producer = KafkaProducer(
    bootstrap_servers=kafka_bootstrap_servers,
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

print(f"Producing messages to Kafka topic '{kafka_topic}'...")

try:
    while True:
        # Generate sensor data
        sensor_data = generate_sensor_data()

        # Publish sensor data to Kafka
        data = np.array([e for e in sensor_data.values()])
        print(f"{data=}")
        producer.send(kafka_topic, data)
```

Al hacer esto obtendremos un error como el siguiente:

Terminal — bash

```
Producing messages to Kafka topic 'water_quality'...
data=array([1.77392590e+09, 2.63006815e+01, 8.27793620e+00,
   7.57000000e+00, 1.05800000e+01])
---------------------------------------------------------
TypeError Traceback (most recent call last)
Cell In[15], line 22
    20 data = np.array([e for e in sensor_data.values()])
    21 print(f"{data=}")
---> 22 producer.send(kafka_topic, data)
    24 print(f"Sent: {sensor_data}")
    26 # Wait for 1 second

TypeError: Object of type ndarray is not JSON serializable
```

Este error indica que el objeto de tipo `ndarray` no puede ser serializado mediante JSON. El motivo es que el serializador configurado (`json.dumps`) solo admite tipos nativos de Python (como `int`, `float`, `str`, `list`, `dict`, etc.), pero no estructuras propias de bibliotecas externas como `NumPy`.

Para resolver este problema existen varias estrategias:

- Convertir explícitamente el array a un tipo serializable (por ejemplo, una lista de Python).
- Definir un serializador personalizado que transforme objetos de `NumPy`.
- Utilizar formatos de serialización alternativos (como Avro o Protobuf) que soporten esquemas más ricos.

2.5 Modificar el tópico para que tenga 2 particiones

Para realizar esta modificación debemos ejecutar el siguiente comando en la bash:

Terminal — bash

```
  root@fa1dbb2ab5cd:/opt# ./kafka/bin/kafka-topics.sh \
 --bootstrap-server localhost:9092 \
 --alter \
 --topic water_quality \
 --partitions 2
root@fa1dbb2ab5cd:/opt#
```

Como podemos observar, ejecutar este comando no revuelve ningún resultado. Por lo que tenemos que verificarlo de la siguiente manera:

```
root@fa1dbb2ab5cd:/opt# kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --topic water_quality
Topic: water_quality    TopicId: rNpDbRzrS32gvjL_-crtaw PartitionCount: 2       ReplicationFactor: 1    Configs:
        Topic: water_quality    Partition: 0    Leader: 0       Replicas: 0     Isr: 0
        Topic: water_quality    Partition: 1    Leader: 0       Replicas: 0     Isr: 0
root@fa1dbb2ab5cd:/opt#
```

Figure 5: Comando y resultados para verificar el número de particiones del tópico.

En la figura 5 se puede observar que *PartitionCount* es ahora 2. Además se puede identificar que tal y como está actualmente configurado, no hay redundancia ya que *ReplicationFactor* = 1. A pesar de haber dos particiones, las dos están lideradas por el mismo *broker*. Esto es debido a que solo hemos configurado un nodo. Es recomendable aumentar el número de productores para habilitar la tolerancia a fallos y distribuir la carga mediante una paralelización real.

2.6 Crear varios productores

Para crear múltiples productores, se han realizado varios cambios. En primer lugar, hemos interpretado *generar varios productores* como instanciar nuevos objetos de la clase *KafkaProducer*; no solo usar una sola instancia y enviar mensajes con *keys* diferentes. Por tanto el proceso consiste en:

```
1. Elegir un numero K de productores
2. Instanciar K productores e IDs y almacenarlos
3. Seleccionar el primer productor y su ID
4. Generar datos sinteticos
5. Enviar los datos sinteticos mediante el productor seleccionado usando como key su ID
6. Seleccionar el siguiente productor en la lista y su ID y repetir los pasos 4 a 5 hasta
    haber seleccionado todos los productores
7. Repetir infinitamente los pasos 3 a 6
```

Los mensajes enviados a un topic con la misma key siempre se asignan a la misma partición. Se asigna el ID del productor a la key a la hora de enviar el mensaje para

lograr que los mensajes de un productor siempre vayan a la misma partición.

Para demostrar que la generación de varios productores ha tenido éxito se adjuntan las siguiente pruebas:

```
Sent: {'timestamp': 1774009172, 'water_temperature': 26.18834365204124, 'ph_level': 8.441345605347603, 'turbidity': 45.62, 'dissolved_oxygen': 10.51, 'producer_id': 'producerID_0'}
Sent: {'timestamp': 1774009172, 'water_temperature': 26.55982156934779, 'ph_level': 7.714538179062096, 'turbidity': 12.21, 'dissolved_oxygen': 10.34, 'producer_id': 'producerID_1'}
Sent: {'timestamp': 1774009172, 'water_temperature': 26.020118311386515, 'ph_level': 8.409899550328602, 'turbidity': 6.42, 'dissolved_oxygen': 11.74, 'producer_id': 'producerID_2'}
Sent: {'timestamp': 1774009172, 'water_temperature': 26.419420928714498, 'ph_level': 7.561661611186286, 'turbidity': 24.86, 'dissolved_oxygen': 6.58, 'producer_id': 'producerID_3'}
Sent: {'timestamp': 1774009172, 'water_temperature': 26.09414976874809, 'ph_level': 8.346212053523077, 'turbidity': 8.47, 'dissolved_oxygen': 10.2, 'producer_id': 'producerID_4'}
Sent: {'timestamp': 1774009172, 'water_temperature': 26.15619723222628, 'ph_level': 7.785736425095738, 'turbidity': 15.09, 'dissolved_oxygen': 8.67, 'producer_id': 'producerID_5'}
Sent: {'timestamp': 1774009172, 'water_temperature': 26.607103718826888, 'ph_level': 7.689282248435438, 'turbidity': 33.98, 'dissolved_oxygen': 9.11, 'producer_id': 'producerID_6'}
Sent: {'timestamp': 1774009173, 'water_temperature': 27.88987768469505, 'ph_level': 8.238460443046753, 'turbidity': 44.25, 'dissolved_oxygen': 7.4, 'producer_id': 'producerID_0'}
Sent: {'timestamp': 1774009173, 'water_temperature': 27.990820571187204, 'ph_level': 7.805017907902941, 'turbidity': 49.68, 'dissolved_oxygen': 7.53, 'producer_id': 'producerID_1'}
Sent: {'timestamp': 1774009173, 'water_temperature': 27.420967565485356, 'ph_level': 7.510421443731092, 'turbidity': 33.55, 'dissolved_oxygen': 9.43, 'producer_id': 'producerID_2'}
Sent: {'timestamp': 1774009173, 'water_temperature': 26.991230358098026, 'ph_level': 7.659787524055387, 'turbidity': 42.88, 'dissolved_oxygen': 11.13, 'producer_id': 'producerID_3'}
Sent: {'timestamp': 1774009173, 'water_temperature': 26.392867140429487, 'ph_level': 8.07590818745918, 'turbidity': 11.59, 'dissolved_oxygen': 10.37, 'producer_id': 'producerID_4'}
Sent: {'timestamp': 1774009173, 'water_temperature': 26.939079267604136, 'ph_level': 7.577406483774405, 'turbidity': 33.71, 'dissolved_oxygen': 8.08, 'producer_id': 'producerID_5'}
Sent: {'timestamp': 1774009173, 'water_temperature': 27.62161027049691, 'ph_level': 8.327339616590582, 'turbidity': 28.58, 'dissolved_oxygen': 5.21, 'producer_id': 'producerID_6'}
Sent: {'timestamp': 1774009175, 'water_temperature': 27.651348113512363, 'ph_level': 8.265103552022586, 'turbidity': 41.24, 'dissolved_oxygen': 11.85, 'producer_id': 'producerID_0'}
```

Figure 6: Al añadir más productores y recorrerlos con un *for* estos envían sus datos de manera secuencial

```
Received: {'timestamp': 1774009183, 'water_temperature': 27.27712563067657, 'ph_level': 7.998333124481641, 'turbidity': 43.07, 'dissolved_oxygen': 5.22, 'producer_id': 'producerID_1'}
Received: {'timestamp': 1774009183, 'water_temperature': 27.056323201815143, 'ph_level': 8.144846894834346, 'turbidity': 18.83, 'dissolved_oxygen': 7.47, 'producer_id': 'producerID_0'}
Received: {'timestamp': 1774009183, 'water_temperature': 27.379022937369704, 'ph_level': 8.266153466037638, 'turbidity': 49.25, 'dissolved_oxygen': 7.03, 'producer_id': 'producerID_4'}
Received: {'timestamp': 1774009183, 'water_temperature': 26.775448569513546, 'ph_level': 7.776544813446375, 'turbidity': 28.75, 'dissolved_oxygen': 10.56, 'producer_id': 'producerID_6'}
Received: {'timestamp': 1774009183, 'water_temperature': 26.741441582904514, 'ph_level': 8.045289237314465, 'turbidity': 40.83, 'dissolved_oxygen': 9.88, 'producer_id': 'producerID_5'}
Received: {'timestamp': 1774009183, 'water_temperature': 26.917480324827554, 'ph_level': 8.109526920600183, 'turbidity': 21.15, 'dissolved_oxygen': 9.11, 'producer_id': 'producerID_3'}
Received: {'timestamp': 1774009183, 'water_temperature': 27.94965848913558, 'ph_level': 8.080396551522117, 'turbidity': 34.39, 'dissolved_oxygen': 6.53, 'producer_id': 'producerID_2'}
Received: {'timestamp': 1774009184, 'water_temperature': 27.2160003802256, 'ph_level': 7.609514136520993, 'turbidity': 30.66, 'dissolved_oxygen': 9.44, 'producer_id': 'producerID_0'}
Received: {'timestamp': 1774009184, 'water_temperature': 26.457427124054988, 'ph_level': 8.359427628186612, 'turbidity': 6.12, 'dissolved_oxygen': 11.13, 'producer_id': 'producerID_2'}
Received: {'timestamp': 1774009184, 'water_temperature': 27.423066662660496, 'ph_level': 8.244608736251203, 'turbidity': 46.82, 'dissolved_oxygen': 6.4, 'producer_id': 'producerID_3'}
Received: {'timestamp': 1774009185, 'water_temperature': 26.786377809411437, 'ph_level': 8.184196433815515, 'turbidity': 40.52, 'dissolved_oxygen': 10.03, 'producer_id': 'producerID_0'}
```

Figure 7: Los consumidores leen los mensajes enviados por cada productor en un orden aleatorio

Esto es debido a que *Kafka* no garantiza un orden global de consumo cuando intervienen múltiples claves y particiones. El orden solo está asegurado dentro de una misma partición y como cada *producer_id* se envía con una *key* diferente, *Kafka* las distribuye en distintas particiones. Posteriormente el consumidor, lee de varias particiones de forma concurrente, lo que provoca que los mensajes se muestren intercalados. No obstante, dentro de cada partición el orden se mantiene.

Las modificaciones que se han llevado a cabo en el fichero de producción son las siguientes:

Python

```python
#####################
# Codigo original:
#####################
# Kafka configuration
kafka_topic = "water_quality"
kafka_bootstrap_servers = ["localhost:9092"]

# Create Kafka producer
# producer = KafkaProducer(
# bootstrap_servers=kafka_bootstrap_servers,
# value_serializer=lambda v: json.dumps(v).encode('utf-8')
# )

###########
# Modificacion para varios productores
###########

producer_number = 7
producers = [] # Aqui almacenaremos nuestros producers
producer_ids = [] # Le damos a cada uno un ID unico
# Create Kafka producer
for i in range(producer_number):
  producer_ids.append("producerID_"+str(i)) # Generacion del ID
  producers.append(KafkaProducer(
      bootstrap_servers=kafka_bootstrap_servers,
      value_serializer=lambda v: json.dumps(v).encode('utf-8')
      )) # Instanciamos el producers


try:
   while True:
      # Iteramos por cada producer (simulacion de multiples fuentes de datos)
      for i, producer in enumerate(producers):
         # Generate sensor data
         sensor_data = generate_sensor_data()

         # Publish sensor data to Kafka
         # Aniadimos identificador al mensaje
         sensor_data["producer_id"] = producer_ids[i]
         # Aniadimos una key unica (producer_id) para identificar el origen del mensaje
```

```python
            producer.send(kafka_topic, key=producer_ids[i].encode('utf-8'), value=sensor_data)

            print(f"Sent: {sensor_data}")

        # Wait for 1 second
        time.sleep(1)
except KeyboardInterrupt:
    print("Stopped producing messages.")
finally:
    # Cerramos todos los producers
    [producer.close() for producer in producers]
```

2.7 Consumidores con ID común

Para poder procesar mensajes con varios consumidores que pertenezcan a un grupo se deben hacer los siguientes cambios:

En primer lugar, debemos tener un almacenamiento que permita agrupar los IDs de sensores de los que vaya a hacerse cargo el consumidor. Para ello vamos a utilizar un diccionario de *python* con una cola que controle el número máximo de elementos que debe almacenar:

Python

```python
MAX_SAMPLES = 100
# Almacenamiento dinamico
data_by_sensor = {}
def process_message(message_value):
    """Extrae y almacena los datos del sensor."""
    s_id = message_value['producer_id']

    # Inicializacion con deque para manejo automatico de tamanio
    if s_id not in data_by_sensor:
        data_by_sensor[s_id] = {
            "timestamps": deque(maxlen=MAX_SAMPLES),
            **{k: deque(maxlen=MAX_SAMPLES) for k in METRICS.keys()}
        }

    storage = data_by_sensor[s_id]
```

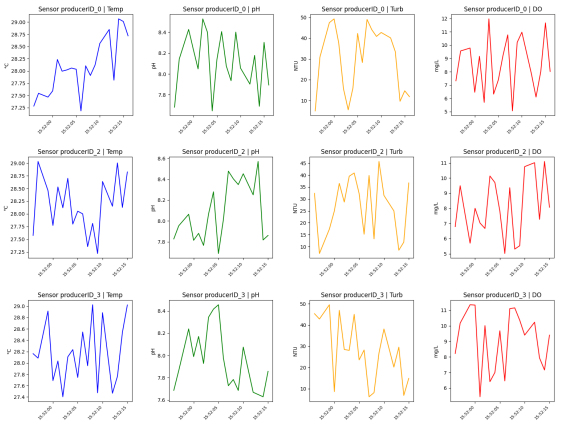
```
    storage["timestamps"].append(message_value['timestamp'])
    for key in METRICS:
        storage[key].append(message_value[key])

    return s_id
```

Además de esto, se debe añadir el ID del grupo al consumidor y cómo deserializar el ID de la key:

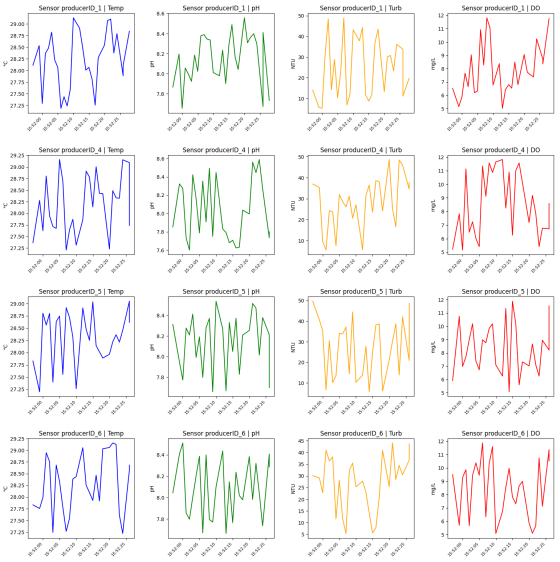
Python

```
    # Kafka configuration
def initialize_consumer():
    kafka_topic = "water_quality"
    kafka_bootstrap_servers = ["localhost:9092"]

    # Create Kafka consumer
    consumer = KafkaConsumer(
        kafka_topic,
        bootstrap_servers=kafka_bootstrap_servers,
        value_deserializer=lambda m: json.loads(m.decode('utf-8')
            ),
        auto_offset_reset='latest',
        enable_auto_commit=True,
        group_id="water_consumers_group", # Aniadimos el
            consumidor a un grupo
        key_deserializer=lambda k: k.decode('utf-8') if k else
            None
    )
    return consumer
```

Una vez hemos hecho esto podemos duplicar el cuadernos para tener dos consumidores escuchando a la vez. Si hacemos estos podemos observar cómo se distribuyen los IDs de las *keys* entre los consumidores: una *key* es asignada exactamente a un consumidor.



(a) Gráfico de calidad del agua del primer consumidor



(b) Gráfico de calidad del agua del segundo consumidor

En la imagen se puede observar claramente cómo se distribuye la carga del procesamiento de las *keys*. Si realizamos estos cambios se puede obtener estos resultados en el cuaderno:

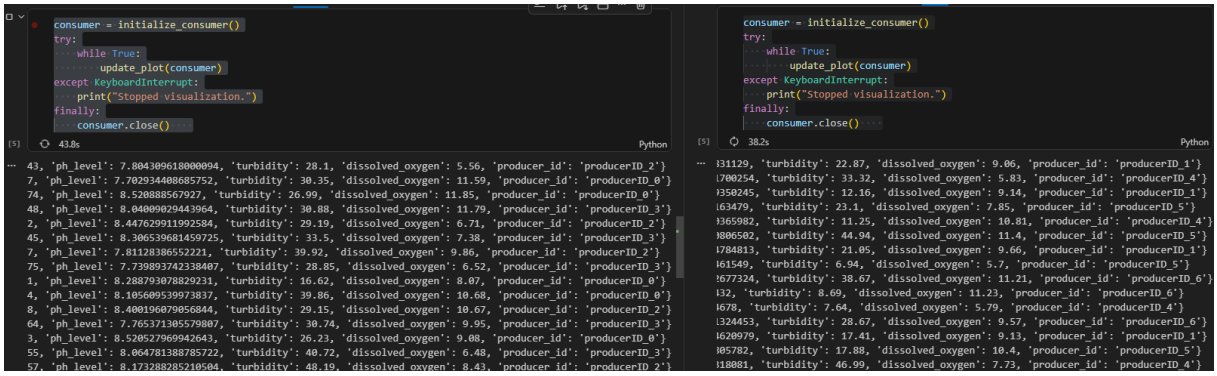


Figure 9: Logs de los dos cuadernos consumiendo a la vez

3 Integrando procesos ML en data streaming

3.1 Carga del modelo e inferencia

Para la implementación del sistema de toma de *Machine Learning*, se ha desarrollado el siguiente código. La función *perform_inference* se encarga de transformar el diccionario de datos del sensor en un vector de características compatible con el modelo, extrayendo las variables de temperatura, pH, turbidez y oxígeno disuelto.

Python

```python
import joblib
import pandas as pd

MODEL_PATH = "aeration_classifier.joblib"

def load_prediction_model(model_uri):
  """Carga el modelo desde el archivo local mediante deserializacion."""
  try:
    model = joblib.load(model_uri)
    return model
  except Exception as e:
    return None

def perform_inference(model, sensor_data):
  """
  Ejecuta la prediccion y el calculo de densidades de probabilidad.
  Retorna la clase predicha y el vector de probabilidades.
  """
  try:
    input_data = [
    sensor_data["water_temperature"],
    sensor_data["ph_level"],
    sensor_data["turbidity"],
    sensor_data["dissolved_oxygen"]
    ]

     prediction = model.predict([input_data])[0]
     probabilities = model.predict_proba([input_data])[0]

     return prediction, probabilities
  except Exception as e:
     return None, None
```

Cada vez que llega un nuevo mensaje al tópico se lleva a cabo una inferencia.

3.2 Envio de predicciones a un nuevo tópico

El siguiente código imprime por pantalla los detalles de la predicción y envia la información solicitada al siguiente consumidor:

Python

```python
for message in consumer:
    sensor_data = message.value
    time_stp = sensor_data["timestamp"]

    # Convertir timestamp a formato legible
    dt = datetime.fromtimestamp(time_stp)
    formatted_time = dt.strftime()

    # Inferencia del modelo
    prediction, probas = perform_inference(model, sensor_data)

    # Etiqueta legible
    label = "NO AERATION" if prediction == 0 else "AERATION"

    # Print limpio
    print(
        f"[{formatted_time}] \t| "
        f"Prediction: {label} ({prediction}) \t| "
        f"Probabilities -> "
        f"NO_AERATION: {probas[0]:.2\%}, "
        f"AERATION: {probas[1]:.2\%}"
    )

    # Envio a Kafka
    send_data = {
        "timestamp": time_stp,
        "prediction": int(prediction),
        "proba_no_areation": float(probas[0]),
        "proba_areation": float(probas[1])
    }

    producer.send(pred_topic, send_data)
```

El resultado de esta celda de código es el siguiente:

```
Subscribed to Kafka topic 'water_quality'.
:D Modelo cargado exitosamente desde aeration_classifier.joblib
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:380: InconsistentVersionWarning: Trying to unpickle estimator
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:380: InconsistentVersionWarning: Trying to unpickle estimator
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
Received: {'timestamp': 1773923099, 'water_temperature': 27.624807720049287, 'ph_level': 8.253830159652212, 'turbidit
[2026-03-19 12:25:00]   | Prediction: NO AERATION (0)   | Probabilities -> NO_AERATION: 90.00%, AERATION: 10.00%
[2026-03-19 12:25:01]   | Prediction: NO AERATION (0)   | Probabilities -> NO_AERATION: 100.00%, AERATION: 0.00%
[2026-03-19 12:25:02]   | Prediction: AERATION (1)      | Probabilities -> NO_AERATION: 0.00%, AERATION: 100.00%
[2026-03-19 12:25:03]   | Prediction: AERATION (1)      | Probabilities -> NO_AERATION: 0.00%, AERATION: 100.00%
[2026-03-19 12:25:04]   | Prediction: NO AERATION (0)   | Probabilities -> NO_AERATION: 100.00%, AERATION: 0.00%
[2026-03-19 12:25:05]   | Prediction: NO AERATION (0)   | Probabilities -> NO_AERATION: 100.00%, AERATION: 0.00%
```

Figure 10: Logs producidos por el consumidor de water data

El siguiente paso es programar un consumidor que reciba los datos y genere la visualización:

👂 Escuchamos

```python
consumer = initialize_consumer()
print("Subscribed to Kafka topic 'water_quality'.")

try:
    while True:
        update_plot(consumer)

except KeyboardInterrupt:
    print("Stopped visualization.")
    consumer.close()
```

```
Subscribed to Kafka topic 'water_quality'.
Received: {'timestamp': 1773918053, 'prediction': 1, 'proba_no_areation': 0.0, 'proba_areation': 1.0}
Received: {'timestamp': 1773918054, 'prediction': 0, 'proba_no_areation': 1.0, 'proba_areation': 0.0}
Received: {'timestamp': 1773918055, 'prediction': 0, 'proba_no_areation': 1.0, 'proba_areation': 0.0}
Received: {'timestamp': 1773918056, 'prediction': 1, 'proba_no_areation': 0.0, 'proba_areation': 1.0}
Received: {'timestamp': 1773918057, 'prediction': 1, 'proba_no_areation': 0.0, 'proba_areation': 1.0}
Received: {'timestamp': 1773918058, 'prediction': 0, 'proba_no_areation': 1.0, 'proba_areation': 0.0}
Received: {'timestamp': 1773918059, 'prediction': 1, 'proba_no_areation': 0.0, 'proba_areation': 1.0}
Received: {'timestamp': 1773918060, 'prediction': 1, 'proba_no_areation': 0.0, 'proba_areation': 1.0}
```

Figure 11: Consumidor de las predicciones

Este nodo genera plots como los siguientes:

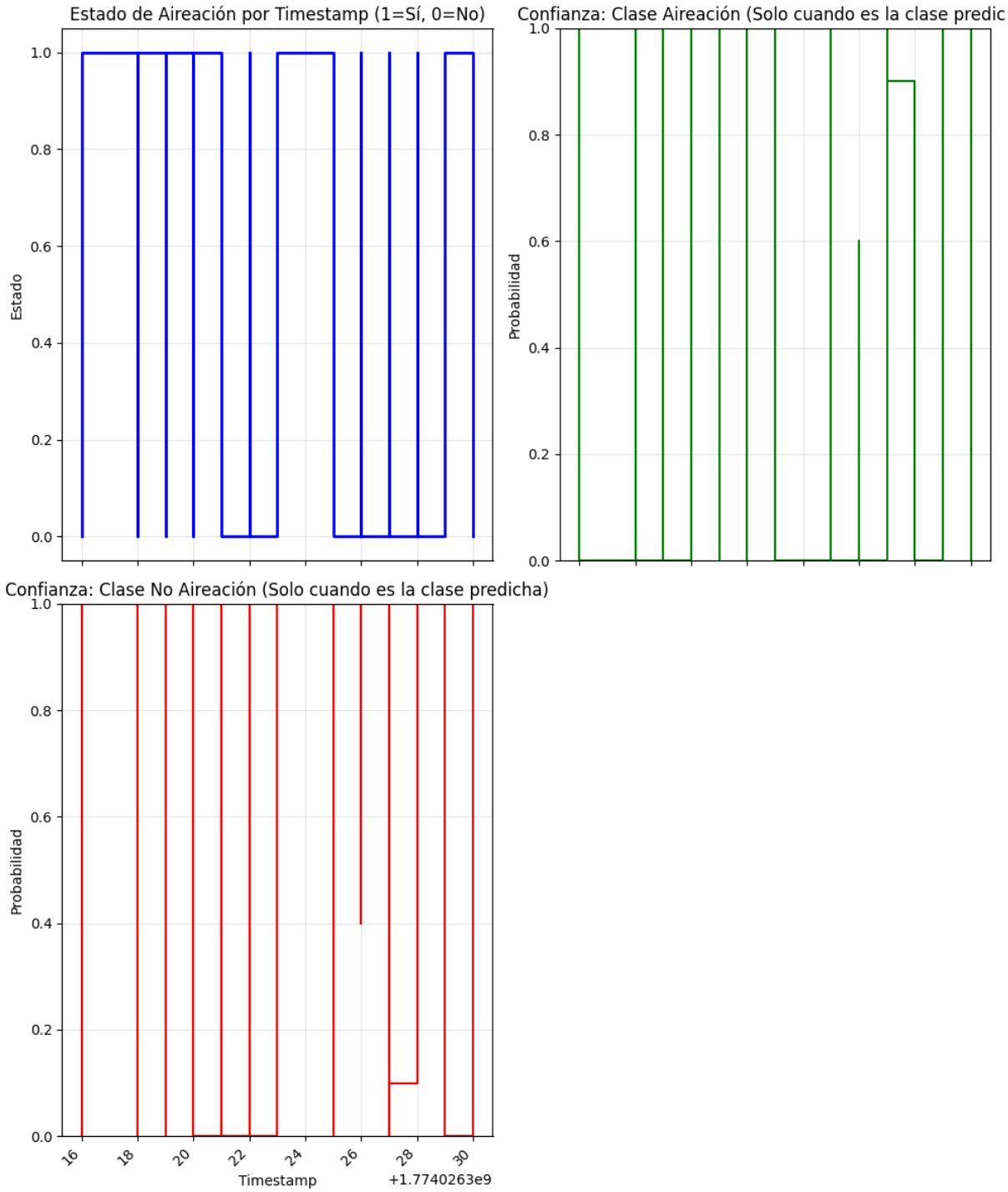


Figure 12: Grafico de predicciones

Si se ejecutan varios procesadores ML el grafico generado es el siguiente:

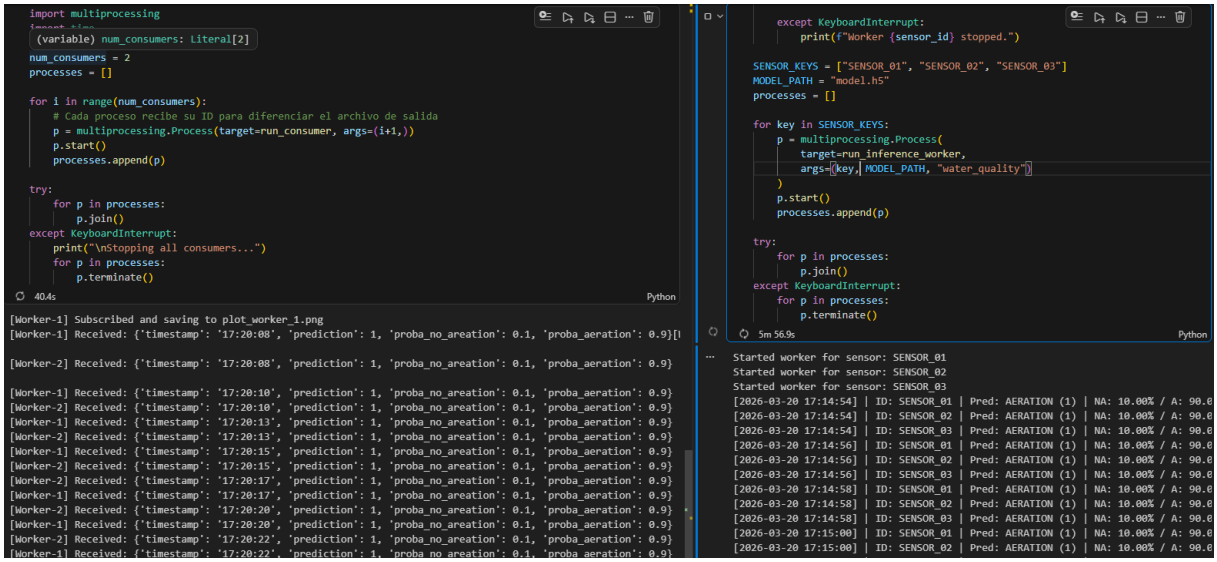


Figure 13: 3 Consumidores del topic water quality realizando la inferencia y enviandola a 2 consumidores. Todo esto se realiza en paralelo